Week 12: Neural Networks

Francisco Richter Ernst Wit

Introduction to Data Science (MSc)

1 Introduction to Neural Networks

Neural networks represent a powerful class of machine learning models inspired by biological neural systems. They provide a flexible framework for approximating complex nonlinear functions and have become fundamental to modern artificial intelligence and data science applications.

Neural networks can be viewed as compositions of simple nonlinear functions. By stacking many layers of these simple transformations, one can approximate complex relationships between inputs and outputs. Learning occurs by adjusting parameters to minimize a loss measuring discrepancy between predictions and observed targets.

2 Mathematical Foundations

2.1 The Perceptron

Definition 2.1 (Perceptron)**.** *A perceptron is a linear classifier that computes:*

$$\hat{y} = sign\left(\sum_{i=1}^{p} w_i x_i + b\right) = sign(\mathbf{w}^T\mathbf{x} + b)$$

where $\mathbf{w}$ *are weights,* b *is the bias, and* $sign(\cdot)$ *is the activation function.*

The perceptron learns a separating hyperplane by iteratively updating the weight vector when a point is misclassified. On linearly separable data, the updates converge to a separating solution. The following illustration shows a learned boundary on a toy two-dimensional dataset, emphasizing the geometry of linear separability.

2.2 Activation Functions

Definition 2.2 (Activation Function)**.** *An activation function* $\sigma : \mathbb{R} \to \mathbb{R}$ *introduces nonlinearity into the network. Common choices include:*

- ***Sigmoid****:* $\sigma(z) = \frac{1}{1+e^{-z}}$
- ***Hyperbolic tangent****:* $\sigma(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- ***ReLU****:* $\sigma(z) = \max(0, z)$
- ***Leaky ReLU****:* $\sigma(z) = \max(\alpha z, z)$ *where* $\alpha \in (0, 1)$

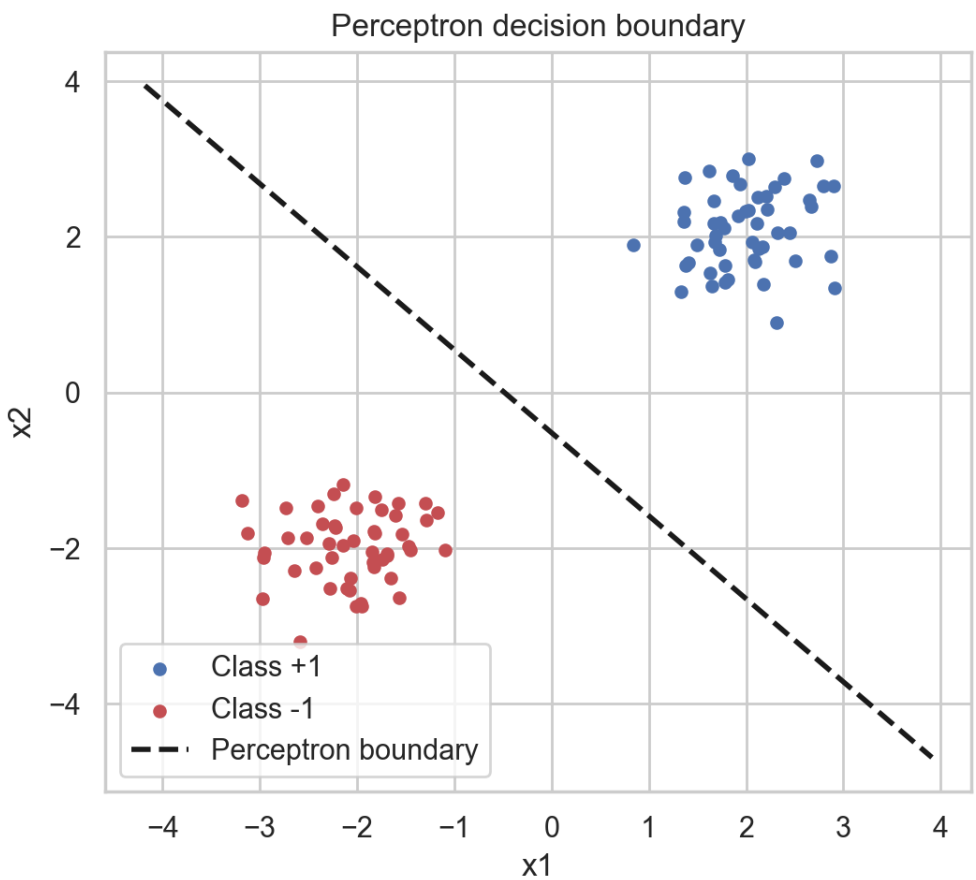

Figure 1: Perceptron decision boundary on a linearly separable dataset.

2.3 Universal Approximation Theorem

Theorem 2.1 (Universal Approximation)**.** *A feedforward neural network with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$ to arbitrary accuracy, provided the activation function is non-constant, bounded, and monotonically increasing.*

3 Feedforward Neural Networks

3.1 Architecture

Definition 3.1 (Multilayer Perceptron (MLP))**.** *An MLP with L layers computes:*

$$\mathbf{a}^{(0)} = \mathbf{x} \quad \textit{(input layer)} \tag{1}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \textit{(linear transformation)} \tag{2}$$

$$\mathbf{a}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)}) \quad \textit{(activation)} \tag{3}$$

for $l = 1, 2, \ldots, L$, where $\mathbf{a}^{(L)}$ is the output.

3.2 Forward Propagation

The forward pass computes the network output given input $\mathbf{x}$:

$$f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{a}^{(L)}$$

where $\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$ are the network parameters.

4 Loss Functions

4.1 Regression Tasks

Definition 4.1 (Mean Squared Error)**.**

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \|\mathbf{y}_i - f(\mathbf{x}_i; \boldsymbol{\theta})\|^2$$

4.2 Classification Tasks

Definition 4.2 (Cross-Entropy Loss)**.** *For binary classification:*

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)]$$

For multi-class classification:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log(\hat{p}_{ik})$$

5 Backpropagation Algorithm

5.1 Chain Rule Application

Backpropagation uses the chain rule to compute gradients efficiently:

Theorem 5.1 (Backpropagation Equations)**.** *For layer l:*

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot \sigma'^{(L)}(\mathbf{z}^{(L)}) \tag{4}$$

$$\boldsymbol{\delta}^{(l)} = [(\mathbf{W}^{(l+1)})^T\boldsymbol{\delta}^{(l+1)}] \odot \sigma'^{(l)}(\mathbf{z}^{(l)}) \tag{5}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}(\mathbf{a}^{(l-1)})^T \tag{6}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \tag{7}$$

where $\odot$ denotes element-wise multiplication.

5.2 Computational Complexity

The computational complexity of backpropagation is $O(W)$ where W is the total number of weights, making it efficient for training large networks.

6 Optimization Methods

Gradient-based learning proceeds by repeated linearization of the objective and movement along the negative gradient direction. In practice, we compute gradients on mini-batches to reduce variance and improve hardware utilization. A simple visualization of a two-parameter loss surface and a gradient descent trajectory clarifies step sizes and curvature effects.

Adaptive methods such as AdaGrad and Adam rescale or accumulate gradients to accommodate heterogeneity across parameters. This can accelerate early learning and stabilize training in the presence of ill-conditioned curvature, though careful tuning and validation remain essential.

7 Regularization Techniques

Controlling generalization error requires explicit complexity control. Weight decay augments the loss with an L2 penalty,

$$\mathcal{L}_{\text{reg}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \lambda\sum_{l=1}^{L}\|\mathbf{W}^{(l)}\|_F^2,$$

shrinking parameter norms toward zero. Dropout introduces multiplicative Bernoulli noise during training, implicitly averaging an ensemble of subnetworks. Batch normalization reduces internal covariate shift by standardizing layer inputs and learning affine corrections.

The effect of these techniques is readily seen on validation curves. As capacity grows or training continues, validation loss initially decreases and then increases. Early stopping selects the epoch that minimizes validation loss.

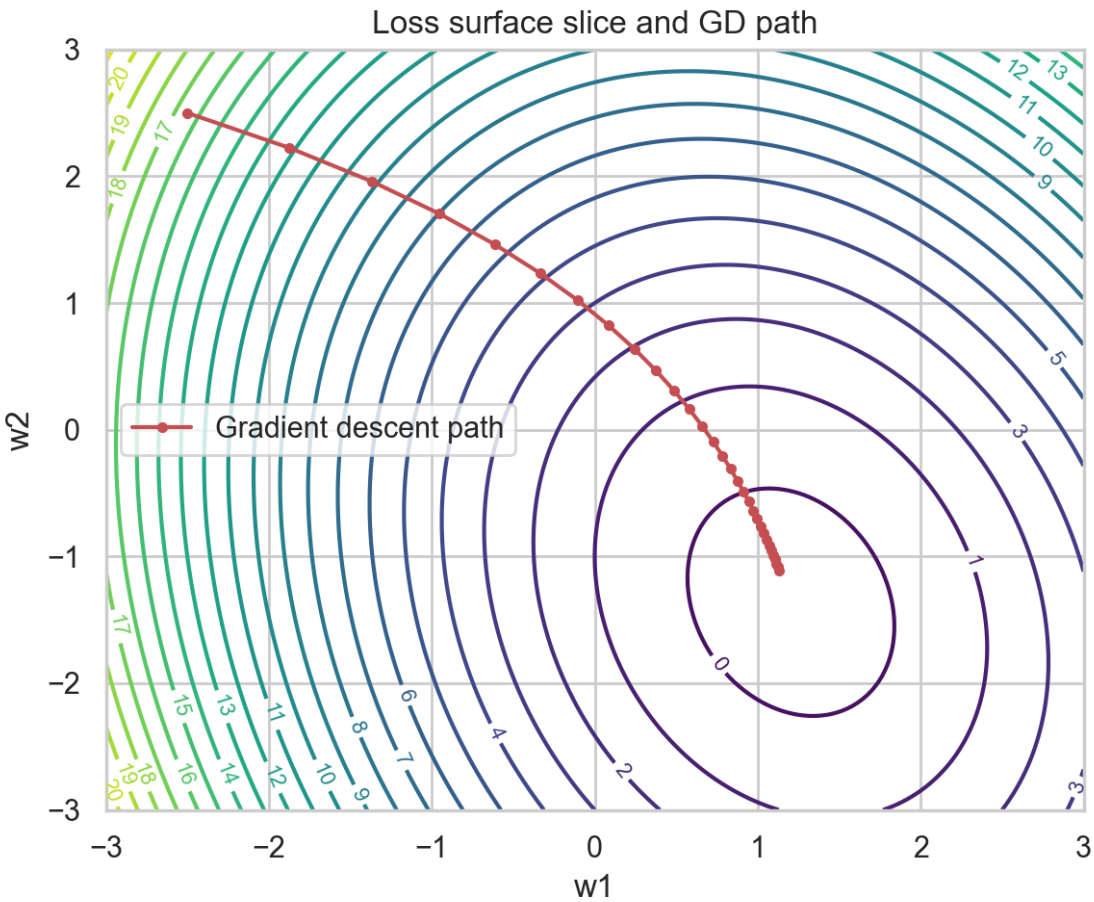

Figure 2: A loss surface slice with a gradient descent trajectory. Step sizes and curvature jointly determine progress.

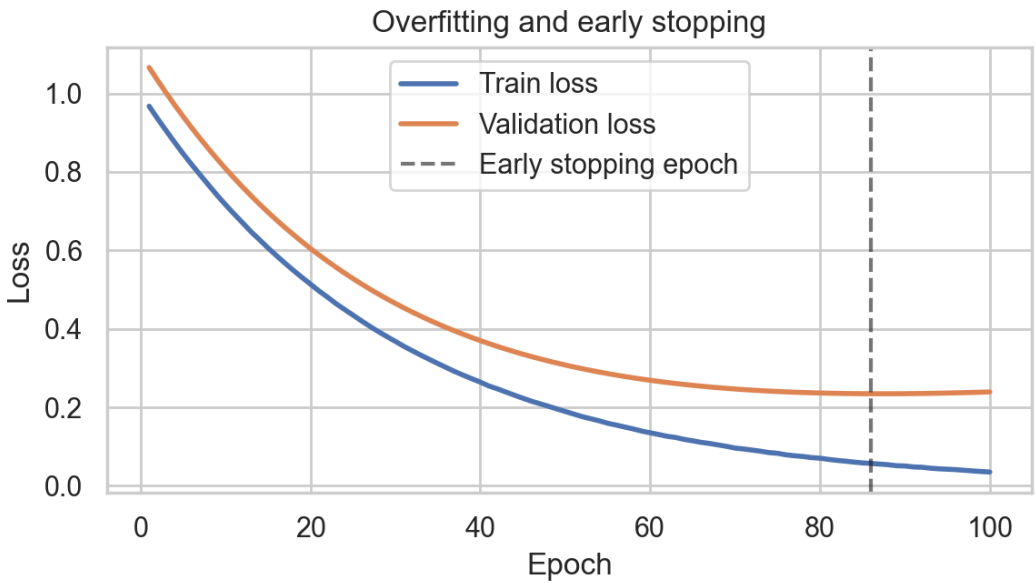

Figure 3: Training versus validation loss with early stopping at the optimal epoch.

8 Network Schematic

To anchor notation, the following schematic depicts a minimal multilayer perceptron and its information flow from inputs, through a hidden layer with ReLU activation, to a sigmoid or softmax output. This diagram will be used to reference weight matrices and activations in the subsequent derivations.

9 High-Dimensional Input Spaces and Representation Learning

9.1 The High-Dimensional Input Problem

Definition 9.1 (High-Dimensional Classification Problem)**.** *Consider input data* $\mathbf{x} \in \mathbb{R}^d$ *where* d *is very large (e.g., image pixels* $d = 28 \times 28 = 784$ *for MNIST, or* $d = 32 \times 32 \times 3 = 3072$ *for CIFAR-10). The goal is to learn a mapping* $f : \mathbb{R}^d \to \{0, 1, \ldots, K-1\}$ *for classification.*

Example 9.1 (Image Recognition Challenge)**.** *For a* 4×4 *grayscale image (16 pixels), we have* $\mathbf{x} = (x_1, x_2, \ldots, x_{16})$ *where each* x_i *represents pixel intensity. The challenge: "Can we identify the letter* π *in a* 4×4 *grid?"*

This represents a typical machine learning question:

- ***Classification task****: Binary classification (*π *vs not* π*)*
- ***High-dimensional input****: 16-dimensional input space*
- ***Minimal semantic meaning****: Individual pixel values have little interpretable meaning*

9.2 Limitations of Linear Models

Theorem 9.1 (Linear Model Limitations in High Dimensions)**.** *Consider logistic regression for the image recognition problem:*

$$Y_i = \begin{cases} 1 & \textit{if image } i \textit{ contains } \pi \\ 0 & \textit{otherwise} \end{cases}$$

$$\mathbf{X}_i = (X_{i1}, X_{i2}, \ldots, X_{i16}) \ \textit{(pixel values)}$$

The logistic regression model:

$$P(Y_i = 1|\mathbf{X}_i) = \frac{1}{1 + \exp(-\mathbf{X}_i^T \boldsymbol{\beta})}$$

This approach has fundamental limitations:

1. ***Linear decision boundaries**: Cannot capture complex spatial patterns*
2. ***No spatial structure**: Treats pixels as independent features*
3. ***No translation invariance**: π in different positions treated as different patterns*

9.3 Automatic Feature Learning

Definition 9.2 (Representation Learning)**.** *Neural networks automatically learn hierarchical feature representations:*

$$\mathbf{h}^{(1)} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \quad \textit{(low-level features)} \tag{8}$$

$$\mathbf{h}^{(2)} = \sigma(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}) \quad \textit{(mid-level features)} \tag{9}$$

$$\vdots \tag{10}$$

$$\mathbf{h}^{(L-1)} = \sigma(\mathbf{W}^{(L-1)}\mathbf{h}^{(L-2)} + \mathbf{b}^{(L-1)}) \quad \textit{(high-level features)} \tag{11}$$

$$\mathbf{y} = \textit{softmax}(\mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}) \quad \textit{(classification)} \tag{12}$$

Each layer $\mathbf{h}^{(\ell)}$ learns increasingly abstract representations:

- ***Layer 1**: Edge detectors, simple patterns*
- ***Layer 2**: Combinations of edges, shapes*
- ***Layer 3**: Object parts, complex patterns*
- ***Output**: Class probabilities*

9.4 Curse of Dimensionality and Neural Networks

Theorem 9.2 (Curse of Dimensionality)**.** *In high-dimensional spaces, several phenomena occur that make traditional methods ineffective:*

1. ***Volume concentration**: In $\mathbb{R}^d$, as $d \to \infty$, the volume of a hypersphere concentrates near its surface.*

2. ***Distance concentration**: For random points $\mathbf{x}_1, \mathbf{x}_2$ in high dimensions:*

$$\lim_{d\to\infty} \frac{\|\mathbf{x}_1 - \mathbf{x}_2\|_2}{\mathbb{E}[\|\mathbf{x}_1 - \mathbf{x}_2\|_2]} = 1$$

3. ***Empty space phenomenon**: The ratio of the volume of a hypersphere to its enclosing hypercube approaches 0 as dimension increases.*

Theorem 9.3 (Neural Networks and Dimensionality)**.** *Neural networks mitigate the curse of dimensionality through:*

1. ***Dimensionality reduction**: Each layer can reduce effective dimensionality by learning relevant features.*
2. ***Manifold learning**: Networks can learn that high-dimensional data lies on lower-dimensional manifolds.*
3. ***Hierarchical feature extraction**: Progressive abstraction reduces dependence on raw input dimensionality.*

9.5 Universal Approximation and Representation Power

Theorem 9.4 (Enhanced Universal Approximation)**.** *Let σ be a non-polynomial activation function. Then:*

1. ***Width sufficiency**: For any continuous function f on a compact set $K \subset \mathbb{R}^d$ and $\epsilon > 0$, there exists a neural network with one hidden layer of finite width that approximates f within ϵ.*
2. ***Depth efficiency**: For functions with compositional structure, deep networks require exponentially fewer parameters than shallow networks to achieve the same approximation accuracy.*

Proof Sketch. The proof uses measure theory and functional analysis:

1. Show that neural networks are dense in $C(K)$ using the Stone-Weierstrass theorem
2. Demonstrate that the set of neural network functions separates points and contains constants
3. Apply approximation theory to establish convergence rates

□

10 Deep Learning Theory

10.1 Expressivity

Theorem 10.1 (Depth vs Width Trade-off)**.** *There exist functions that can be represented by a deep network with polynomial width but require exponential width when represented by a shallow network.*

10.2 Optimization Landscape

Theorem 10.2 (Loss Surface Properties)**.** *For sufficiently wide neural networks:*

- *All local minima are global minima*
- *The number of saddle points grows exponentially with network size*
- *Gradient descent can escape saddle points efficiently*

10.3 Generalization Theory

10.3.1 PAC-Bayes Bounds

Theorem 10.3 (PAC-Bayes Generalization Bound)**.** *With probability at least* $1 - \delta$*:*

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta})] \leq \hat{\mathcal{L}}(\boldsymbol{\theta}) + \sqrt{\frac{KL(\rho||\pi) + \log(2\sqrt{n}/\delta)}{2n}}$$

where ρ *is the posterior and* π *is the prior over parameters.*

11 Specialized Architectures

11.1 Convolutional Neural Networks (CNNs)

Definition 11.1 (Convolution Operation)**.**

$$(\mathbf{f} * \mathbf{g})[i, j] = \sum_m \sum_n \mathbf{f}[m, n]\mathbf{g}[i - m, j - n]$$

CNNs use:

- **Local connectivity**: Neurons connect only to local regions
- **Parameter sharing**: Same weights used across spatial locations
- **Translation invariance**: Features detected regardless of position

11.2 Recurrent Neural Networks (RNNs)

Definition 11.2 (RNN Update Equations)**.**

$$\mathbf{h}_t = \sigma(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$
$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

11.2.1 Long Short-Term Memory (LSTM)

LSTM addresses vanishing gradients through gating mechanisms:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{13}$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)} \tag{14}$$
$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad \text{(candidate values)} \tag{15}$$
$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad \text{(cell state)} \tag{16}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output gate)} \tag{17}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad \text{(hidden state)} \tag{18}$$

12 Training Challenges

12.1 Vanishing and Exploding Gradients

In deep chains of nonlinearities, repeated multiplication by Jacobians may shrink or blow up gradient norms. A simple bound illustrates this behavior through products of layer operator norms and activation derivatives. Practice counters these issues by matching initialization to nonlinearity (e.g., Xavier for tanh, He for ReLU), introducing identity-like skip connections that shorten gradient paths (residual networks), clipping gradients to cap extreme steps, and standardizing pre-activations within mini-batches using batch normalization. Each intervention targets a different part of the computation graph, collectively stabilizing learning dynamics.

12.2 Overfitting

Neural networks can overfit when model capacity outstrips sample size or when training proceeds past the point of optimal generalization. Early stopping halts optimization at the epoch that minimizes validation loss; weight decay biases parameters toward smaller norms; data augmentation injects invariances directly into the training distribution; cross-validation estimates out-of-sample error for robust hyperparameter choice.

13 Case Study: MNIST Handwritten Digits

The MNIST dataset consists of grayscale 28×28 images of handwritten digits 0 through 9. Each image is treated as a vector $\mathbf{x} \in \mathbb{R}^{784}$ after flattening, with label $y \in \{0, \ldots, 9\}$. The goal is to learn a function mapping inputs to class probabilities that generalizes beyond the training set.

As a baseline, multinomial logistic (softmax) regression models class probabilities via linear scores $\mathbf{s}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}$ and $\mathrm{softmax}(\mathbf{s})$. Training minimizes empirical cross-entropy; the gradient equals predictions minus one-hot labels times inputs. Identifiability is ensured by fixing a reference class.

Gradient derivation. For a single example $(\mathbf{x}, y)$, let $\mathbf{s} = \mathbf{W}\mathbf{x} + \mathbf{b}$ and $\hat{\boldsymbol{\pi}} = \mathrm{softmax}(\mathbf{s})$. The per-example loss is $\ell = -\log \hat{\boldsymbol{\pi}}_y$. Using $\partial \ell / \partial s_k = \hat{\boldsymbol{\pi}}_k - \mathbf{1}\{k = y\}$, the chain rule gives

$$\nabla_{\mathbf{W}} \ell = (\hat{\boldsymbol{\pi}} - \mathbf{e}_y)\, \mathbf{x}^T, \qquad \nabla_{\mathbf{b}} \ell = \hat{\boldsymbol{\pi}} - \mathbf{e}_y,$$

and averaging over the sample yields the earlier expressions. The key step is the Jacobian of softmax, $\partial \hat{\boldsymbol{\pi}}_k / \partial s_j = \hat{\boldsymbol{\pi}}_k (\mathbf{1}\{j = k\} - \hat{\boldsymbol{\pi}}_j)$.

Numerical stability matters in practice: rather than computing $\log \sum_j \exp(s_j)$ directly, subtract the maximum logit $m = \max_j s_j$ and use $\log \sum_j \exp(s_j - m) + m$. This "log-sum-exp" trick avoids overflow while leaving softmax probabilities and the loss unchanged because adding a constant to all logits does not affect the distribution.

To increase expressivity, a one-hidden-layer MLP replaces linear scores with $\mathbf{s} = \mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, with ReLU activation $\sigma(z) = \max(0, z)$. Gradients follow from backpropagation: the output error is $\mathrm{softmax}(\mathbf{s}) - \mathbf{e}_y$, the hidden error gates with σ'.

Preprocessing rescales pixels to $[0, 1]$ (and optionally standardizes) to improve conditioning. A train/validation/test protocol supports principled model selection (hidden width, learning rate, weight decay) and final evaluation on held-out data. Early stopping and weight decay provide effective complexity control in practice.

13.1 Regularization Comparison: Early Stopping versus Weight Decay

On MNIST, both approaches reduce effective capacity but via different mechanisms. Early stopping halts optimization when validation loss is minimized, implicitly constraining the trajectory length in parameter space; weight decay explicitly penalizes large weights, biasing the solution toward smaller-norm parameters and improving Hessian conditioning. Empirically, early stopping often preserves sharp discriminative features learned early, while L_2 shrinkage smooths decision boundaries. The training–validation curves above illustrate a typical selection of the optimal epoch for early stopping.

14 Applications in Data Science

In computer vision, convolutional architectures learn localized features such as edges and textures that compose into object parts and categories. In natural language processing, recurrent and transformer-based models capture long-range dependencies and contextual meaning, enabling translation and question answering. For time series, neural architectures model nonlinear dynamics for forecasting and anomaly detection. Recommendation systems leverage embeddings that position users and items in a latent space where proximity reflects preference.

As a concrete mental model, consider a small fully connected network mapping pixel intensities to class probabilities. Nonlinear hidden layers transform raw inputs into progressively abstract features; a final sigmoid or softmax converts the last linear scores into calibrated probabilities. Training reduces cross-entropy on mini-batches, while a validation set guides early stopping and regularization strength.

From a generalization perspective, minimizing empirical cross-entropy under appropriate regularization controls expected risk by balancing approximation and estimation errors. Early stopping and weight decay act as capacity control; data augmentation encodes invariances to reduce effective complexity.

15 Summary

Key concepts include understanding the forward and backward propagation processes, the role of different activation functions, and the challenges of training deep networks such as vanishing gradients and overfitting. The connection between neural networks and statistical learning theory provides important theoretical insights.

Practical considerations such as network architecture design, regularization techniques, and optimization algorithms are essential for successful application in data science projects. Understanding neural networks is crucial as they form the foundation for advanced deep learning methods and modern AI applications.

16 Exercises

1. MNIST softmax regression. Derive the gradient of the multinomial logistic loss with respect to $\mathbf{W}$ and $\mathbf{b}$ and show that the estimator is identifiable up to an additive constant only if one class is fixed as reference or a constraint is imposed on the scores. Prove convexity of the loss.

2. One-hidden-layer MNIST classifier. For ReLU hidden units and softmax output, derive the specialized backpropagation equations from the general chain rule and verify dimensions of each Jacobian. Explain how zero-derivative regions affect learning.

3. Generalization via early stopping. Consider training and validation losses as functions of epochs. Formulate an oracle stopping time that minimizes expected risk and argue why validation minimization is a consistent proxy under standard assumptions.

4. Regularization paths. Show that adding an L2 penalty to the softmax regression yields ridge-regularized estimates. Discuss the effect on the Hessian's conditioning and provide bounds on parameter shrinkage as a function of λ.

5. Expressivity. Construct a digit-like synthetic pattern and prove that a network with one hidden layer and ReLU units can represent an indicator of a finite union of axis-aligned rectangles; conclude universal approximation on compact sets by partition refinement.

17 References

Goodfellow, Bengio, and Courville. Deep Learning. MIT Press, 2016.

Bishop. Pattern Recognition and Machine Learning. Springer, 2006.

Hastie, Tibshirani, and Friedman. The Elements of Statistical Learning. Springer, 2009.